

Tighter security for XMSS (Reduction-Based Approach)

Work in Progress

Abstract

Contents

1	Introduction	2
2	Chain Preimage Resistance	2
3	Adaptation of the security proof of Generalized XMSS	3
4	Analysis of SM-CHAIN	4
4.1	Game.2 in the ROM	6

DRAFT

1 Introduction

We attempt to reproduce the proof strategy of [1] (we reuse all its notations and constructions), with the following change in the security proof: Replace the final guessing argument (that uses SM-PRE and SM-UD, which leads to the $L \cdot v \cdot 2^w$ factor overhead) by a new (non-standard ¹) property: Multi-Target Chain Preimage Resistance (SM-CHAIN).

We finally provide a bound for SM-CHAIN in the ROM.

TODO: SM-CHAIN in the QROM

2 Chain Preimage Resistance

Definition 2.1 (Multi-Target Chain Preimage Resistance). Let $\text{Th} : \mathcal{P} \times \mathcal{T} \times \mathcal{M} \rightarrow \mathcal{H}$ be a tweakable hash function with $\mathcal{H} \subseteq \mathcal{M}$. Let $w \geq 1$ be the log-chain-length, let $p \in \mathbb{N}$ be a query bound, and let $\mathcal{A}$ be a stateful algorithm. Consider the experiment $\text{SM-CHAIN}_{\text{Th},w,p}(\mathcal{A})$:

Setup:

1. Sample $P \xleftarrow{\$} \mathcal{P}$.
2. Initialize $Q := \emptyset$, $\text{Chains} := []$, and counter $\text{ctr} := 0$.

Query Phase: Run $\mathcal{A}$ with (classical) access to the Chain Oracle $\mathcal{O}_{\text{chain}}(T_1, \dots, T_{2^w-1})$ for $(T_1, \dots, T_{2^w-1}) \in \mathcal{T}^{2^w-1}$:

- (a) If $\text{ctr} \geq p$ or $|\{T_1, \dots, T_{2^w-1}\}| < 2^w - 1$ or $\{T_1, \dots, T_{2^w-1}\} \cap Q \neq \emptyset$, return $\perp$.
- (b) Sample $Y_0 \xleftarrow{\$} \mathcal{H}$.
- (c) Compute $Y_j := \text{Th}(P, T_j, Y_{j-1})$ for $j = 1, \dots, 2^w - 1$.
- (d) Append $((T_1, \dots, T_{2^w-1}), (Y_0, \dots, Y_{2^w-1}))$ to Chains .
- (e) Set $Q := Q \cup \{T_1, \dots, T_{2^w-1}\}$ and $\text{ctr} := \text{ctr} + 1$.
- (f) Return Y_{2^w-1} .

Reveal Phase: When $\mathcal{A}$ signals to proceed:

1. Disable $\mathcal{O}_{\text{chain}}$ and provide P to $\mathcal{A}$.
2. For each $j \in [|\text{Chains}|]$, initialize $\text{Revealed}_j := \emptyset$.
3. Give $\mathcal{A}$ (classical) access to the Reveal Oracle $\mathcal{O}_{\text{reveal}}(j, k)$ for $j \in [|\text{Chains}|]$, $k \in \{0, \dots, 2^w - 1\}$:
 - (a) Let $(Y_0^j, \dots, Y_{2^w-1}^j)$ be the chain values for chain j .
 - (b) Set $\text{Revealed}_j := \text{Revealed}_j \cup \{k\}$.
 - (c) Return Y_k^j .

Output: $\mathcal{A}$ outputs (j^*, x) with $j^* \in [|\text{Chains}|]$ and $x \in \mathcal{H}$.

Winning Condition: Let $((T_1^{j^*}, \dots, T_{2^w-1}^{j^*}), (Y_0^{j^*}, \dots, Y_{2^w-1}^{j^*}))$ be the j^* th entry in Chains . Let $k^* := \min(\text{Revealed}_{j^*} \cup \{2^w - 1\})$ be the earliest revealed position (or $2^w - 1$ if nothing was revealed). Output 1 if and only if $k^* \neq 0$ and:

$$\text{Th}(P, T_{k^*}^{j^*}, x) = Y_{k^*}^{j^*}.$$

¹The resulting security proof is thus not in the Standard Model (but ROM/QROM)

The advantage is:

$$\text{Adv}_{\text{Th},w,p}^{\text{SM-CHAIN}}(\mathcal{A}) := \Pr[\text{SM-CHAIN}_{\text{Th},w,p}(\mathcal{A}) \Rightarrow 1].$$

Intuition 2.2. The challenger constructs secret hash chains of the form:

$$Y_0 \xrightarrow{T_1} Y_1 \xrightarrow{T_2} Y_2 \xrightarrow{T_3} \dots \xrightarrow{T_{2^w-1}} Y_{2^w-1}$$

but initially only reveals the *final* value Y_{2^w-1} . Later, the attacker may query to see some intermediate values. The attacker wins by finding a preimage of the earliest revealed value in the chain.

3 Adaptation of the security proof of Generalized XMSS

Theorem 3.1 (Security of Generalized XMSS). *Consider the scheme $\text{SIG}[\text{IncEnc}, \text{Th}, K]$ with parameters $n, v, w, L, K \in \mathbb{N}$ and Th, IncEnc as in Construction 3. Then, for every algorithm $\mathcal{A}$, that makes no more than q_s signature queries, there are algorithms $\mathcal{B}_i$ with $T(\mathcal{A}) \approx T(\mathcal{B}_i)$ for all i and*

$$\begin{aligned} & \text{Adv}_{\text{SIG}[\text{IncEnc}, \text{Th}]}^{\text{SY-UF-CMA}}(\mathcal{A}) \\ & \leq \text{Adv}_{\text{Th}, 2 \cdot L \cdot v \cdot 2^w}^{\text{SM-TCR}}(\mathcal{B}_1) + \text{Adv}_{\text{IncEnc}, q_s}^{\text{T-COLL-RES}, K}(\mathcal{B}_2) + \text{Adv}_{\text{Th}, L \cdot v \cdot 2^w}^{\text{SM-TCR}}(\mathcal{B}_3) + 2 \cdot \text{Adv}_{\text{Th}, w, L \cdot v}^{\text{SM-CHAIN}}(\mathcal{B}_4). \end{aligned}$$

Proof. Up to Game.4, the proof is identical to [1] and is omitted here for brevity.

...

From now on, we can hence assume that

$$\text{Chain}_{\text{Th}, i^*, \text{ep}^*}(P, x_{i^*}^*, x_{i^*}^* - x_{i^*}^*, y_{i^*}^*) = y_{i^*}^*,$$

Game.5 (Final Reduction to Chain Preimage Resistance): We now complete the proof by reducing to multi-target chain preimage resistance. Recall that if Game.3 outputs 1, we have $x \neq x^*$ and there exists $j^* = \min\{j \in [v] : x_j^* < x_j\}$.

We construct a reduction $\mathcal{B}_4$ against $\text{SM-CHAIN}_{\text{Th}, w, L \cdot v}$ as follows:

1. **Query phase:** For each epoch $\text{ep} \in [L]$ and chain index $j \in [v]$, query $\mathcal{O}_{\text{chain}}(T_1^{j, \text{ep}}, \dots, T_{2^w-1}^{j, \text{ep}})$ to receive the final chain value $\text{pk}_{\text{ep}, j}$. Store the mapping from (ep, j) to the corresponding chain index in Chains . This uses $L \cdot v$ chain queries.
2. **After reveal phase:** Receive P from the game. Compute the Merkle tree from the one-time public keys $\text{pk}_{\text{ep}} = (\text{pk}_{\text{ep}, 1}, \dots, \text{pk}_{\text{ep}, v})$ for $\text{ep} \in [L]$ to obtain root . Set $\text{pk} := (\text{root}, P)$ and provide pk to $\mathcal{A}$.
3. **Simulating $\mathcal{O}_{\text{Sig}}(\text{ep}, m)$:**
 - (a) Compute the encoding $x = \text{IncEnc}(P, m, \rho, \text{ep})$ by sampling ρ appropriately (if after K attempts, no ρ is found, return $\perp$).
 - (b) For each $j \in [v]$: query $\mathcal{O}_{\text{reveal}}(\text{chain index for } (\text{ep}, j), x_j)$ to obtain y_j , the chain value at position x_j .
 - (c) Compute the Merkle path path_{ep} and return $\sigma := (\rho, (y_1, \dots, y_v), \text{path}_{\text{ep}})$.
4. **Extracting the forgery:** When $\mathcal{A}$ outputs $(\text{ep}^*, m^*, \sigma^*)$ with $\sigma^* = (\rho^*, (y_1^*, \dots, y_v^*), \text{path}_{\text{ep}^*}^*)$:
 - (a) Compute $x^* = \text{IncEnc}(P, m^*, \rho^*, \text{ep}^*)$ and find $j^* = \min\{j : x_j^* < x_j\}$.
 - (b) Compute $z' := \text{Chain}_{\text{Th}, j^*, \text{ep}^*}(P, x_{j^*}^*, x_{j^*}^* - x_{j^*}^* - 1, y_{j^*}^*)$.
 - (c) Output (chain index for $(\text{ep}^*, j^*), z'$).

The simulation is perfect, $T(\mathcal{B}_4) \approx T(\mathcal{A})$, and $\mathcal{B}_4$ wins the SM-CHAIN game whenever Game.3 outputs 1. Therefore:

$$\text{Adv}_{\text{SIG}}^{\text{Game.3}}(\mathcal{A}) \leq \text{Adv}_{\text{Th},w,L \cdot v}^{\text{SM-CHAIN}}(\mathcal{B}_4).$$

Conclusion: Summing over all game hops, we obtain the desired result. $\square$

4 Analysis of SM-CHAIN

Consider the simpler game, denoted Game.1, in which the public parameter P is sent to the adversary directly after being sampled by the challenger (and no longer at the end of the reveal phase). We denote the advantage of an adversary $\mathcal{B}$ in this game by $\text{Adv}_{w,p}^{\text{Game.1}}(\mathcal{B})$. By trivial reduction, we have:

Lemma 4.1. *For any classical (resp. quantum) adversary $\mathcal{A}$ against SM-CHAIN, there exists a classical (resp. quantum) adversary $\mathcal{B}$ against Game.1 with $T(\mathcal{A}) \approx T(\mathcal{B})$ such that:*

$$\text{Adv}_{\text{Th},w,p}^{\text{SM-CHAIN}}(\mathcal{A}) \leq \text{Adv}_{\text{SM-CHAIN},w,p}^{\text{Game.1}}(\mathcal{B}).$$

Consider now Game.2, similar to Game.1 but in which the adversary doesn't have access to the Reveal Oracle. Furthermore, the chain length κ is no longer necessarily a power of two, minus one (previously it was of the form $2^w - 1$). Finally, we introduce a Corruption Oracle that enables the adversary to reveal all the hidden values of a chain, which naturally removes it from the candidates in the Winning Condition. For clarity, we describe Game.2 in full detail below.

Definition 4.2 (Game.2). Let $\text{Th} : \mathcal{P} \times \mathcal{T} \times \mathcal{M} \rightarrow \mathcal{H}$ be a tweakable hash function with $\mathcal{H} \subseteq \mathcal{M}$. Let $\kappa \in [1, 2^w - 1]$ be the chain-length, let $p \in \mathbb{N}$ be a query bound, and let $\mathcal{A}$ be a stateful algorithm. Consider the experiment $\text{GAME-2}_{\text{Th},\kappa,p}(\mathcal{A})$:

Setup:

1. Sample $P \xleftarrow{\$} \mathcal{P}$.
2. Initialize $Q := \emptyset$, $\text{Chains} := []$, $\text{Corrupted} := \emptyset$, and counter $\text{ctr} := 0$.

Query Phase: Run $\mathcal{A}$ with P and (classical) access to:

Chain Oracle $\mathcal{O}_{\text{chain}}(T_1, \dots, T_\kappa)$ for $(T_1, \dots, T_\kappa) \in \mathcal{T}^\kappa$:

- (a) If $\text{ctr} \geq p$ or $|\{T_1, \dots, T_\kappa\}| < \kappa$ or $\{T_1, \dots, T_\kappa\} \cap Q \neq \emptyset$, return $\perp$.
- (b) Sample $Y_0 \xleftarrow{\$} \mathcal{H}$.
- (c) Compute $Y_j := \text{Th}(P, T_j, Y_{j-1})$ for $j = 1, \dots, \kappa$.
- (d) Append $(T_\kappa, (Y_0, \dots, Y_\kappa))$ to Chains .
- (e) Set $Q := Q \cup \{T_1, \dots, T_\kappa\}$ and $\text{ctr} := \text{ctr} + 1$.
- (f) Return Y_κ .

Corruption Oracle $\mathcal{O}_{\text{corrupt}}(j)$ for $j \in [|\text{Chains}|]$:

- (a) Let $(T_\kappa^j, (Y_0^j, \dots, Y_\kappa^j))$ be the j th entry in Chains .
- (b) Set $\text{Corrupted} := \text{Corrupted} \cup \{j\}$.
- (c) Return $(Y_0^j, \dots, Y_\kappa^j)$.

Output: $\mathcal{A}$ outputs (j^*, x) with $j^* \in [\text{Chains}]$ and $x \in \mathcal{H}$.

Winning Condition: Let $(T_\kappa^{j^*}, (Y_0^{j^*}, \dots, Y_\kappa^{j^*}))$ be the j^* th entry in Chains. Output 1 if and only if $j^* \notin \text{Corrupted}$ and:

$$\text{Th}(P, T_\kappa^{j^*}, x) = Y_\kappa^{j^*}.$$

The advantage is:

$$\text{Adv}_{\text{Th}, \kappa, p}^{\text{GAME-2}}(\mathcal{A}) := \Pr[\text{GAME-2}_{\text{Th}, \kappa, p}(\mathcal{A}) \Rightarrow 1].$$

Lemma 4.3. *For any classical (resp. quantum) adversary $\mathcal{A}$ against $\text{GAME-1}_{\text{Th}, w, p}$, and for each κ , there exists $2^w - 1$ classical (resp. quantum) adversaries $(\mathcal{B}_\kappa)_{\kappa \in [1, 2^w - 1]}$, respectively against $(\text{GAME-2}_{\text{Th}, \kappa, p})_{\kappa \in [1, 2^w - 1]}$ with $T(\mathcal{A}) \approx T(\mathcal{B}_\kappa) \forall \kappa \in [1, 2^w - 1]$ such that:*

$$\text{Adv}_{\text{Th}, w, p}^{\text{GAME-1}}(\mathcal{A}) \leq 2^w \cdot \max_{\kappa \in [1, 2^w - 1]} \text{Adv}_{\text{Th}, \kappa, p}^{\text{GAME-2}}(\mathcal{B}_\kappa)$$

Proof. At the start of the reduction, sample $\kappa \xleftarrow{\$} [1, 2^w - 1]$ uniformly at random (hidden from $\mathcal{A}$). We construct the adversary $\mathcal{B}_\kappa$ against $\text{GAME-2}_{\text{Th}, \kappa, p}$ as follows:

Setup: $\mathcal{B}_\kappa$ receives the public parameter P from the GAME-2 challenger and forwards it to $\mathcal{A}$.

Simulating the Chain Oracle: When $\mathcal{A}$ queries $\mathcal{O}_{\text{chain}}(T_1, \dots, T_{2^w-1})$:

- (a) $\mathcal{B}_\kappa$ queries its own chain oracle with $(T_1, \dots, T_\kappa)$ to receive Y_κ .
- (b) $\mathcal{B}_\kappa$ extends the chain by computing:

$$Y_j := \text{Th}(P, T_j, Y_{j-1}) \quad \text{for } j = \kappa + 1, \dots, 2^w - 1.$$

- (c) $\mathcal{B}_\kappa$ stores $(T_1, \dots, T_{2^w-1})$ and $(Y_\kappa, \dots, Y_{2^w-1})$.
- (d) $\mathcal{B}_\kappa$ returns Y_{2^w-1} to $\mathcal{A}$.

Simulating the Reveal Oracle: When $\mathcal{A}$ queries $\mathcal{O}_{\text{reveal}}(j, k)$:

- (a) If $k \geq \kappa$: $\mathcal{B}_\kappa$ retrieves the stored $Y_k^{(j)}$ and returns it to $\mathcal{A}$.
- (b) If $k < \kappa$: $\mathcal{B}_\kappa$ queries $\mathcal{O}_{\text{corrupt}}(j)$ to obtain all chain values $(Y_0^{(j)}, \dots, Y_\kappa^{(j)})$, stores them, and returns $Y_k^{(j)}$ to $\mathcal{A}$. (Subsequent reveal queries on chain j can use these stored values.)

Extracting the Forgery: When $\mathcal{A}$ outputs (j^*, x) :

- (a) Let $k^* := \min(\text{Revealed}_{j^*} \cup \{2^w - 1\})$ be the earliest revealed position for chain j^* .
- (b) If $k^* \neq \kappa$: $\mathcal{B}_\kappa$ **aborts** (wrong guess for κ).
- (c) If $k^* = \kappa$: $\mathcal{B}_\kappa$ outputs (j^*, x) to its GAME-2 challenger.

The simulation of the game provided by the reduction is perfect, and both adversaries have similar running times.

When $\mathcal{A}$ wins $\text{GAME-1}_{\text{Th}, w, p}$ with some value $k^* \in [1, 2^w - 1]$, since κ was chosen uniformly at random (and has no influence on $\mathcal{A}$'s view), the probability that $\kappa = k^*$ is $1/(2^w - 1) \geq 1/2^w$. In this case, $\mathcal{B}_\kappa$ does not abort and wins $\text{GAME-2}_{\text{Th}, \kappa, p}$. Therefore:

$$\frac{1}{2^w} \cdot \text{Adv}_{\text{Th}, w, p}^{\text{GAME-1}}(\mathcal{A}) \leq \mathbb{E}_\kappa \left[\text{Adv}_{\text{Th}, \kappa, p}^{\text{GAME-2}}(\mathcal{B}_\kappa) \right] \leq \max_{\kappa \in [1, 2^w - 1]} \text{Adv}_{\text{Th}, \kappa, p}^{\text{GAME-2}}(\mathcal{B}_\kappa).$$

□

4.1 Game.2 in the ROM

We assume that $\mathcal{A}$ is classical and that Th is modeled as a random oracle.

Lemma 4.4. *Let $\mathcal{A}$ be an adversary against $\text{GAME-2}_{\text{Th}, \kappa, p}$ making at most q_H random oracle queries. If $|\mathcal{H}| > q_H$, then:*

$$\text{Adv}_{\text{Th}, \kappa, p}^{\text{GAME-2}}(\mathcal{A}) \leq \frac{2q_H + 2}{|\mathcal{H}| - q_H}.$$

Proof. Step 1: Notation and definitions.

Let s denote the number of queries (RO, Chain Oracle, or Corruption Oracle) at any point, and s_f the total at game's end. Let $\text{Chains}(s)$ and $\text{Corrupted}(s) \subseteq \text{Chains}(s)$ denote the chains created and corrupted after s queries, respectively.

For $s \in [0, s_f]$, non-corrupted chain $j \in \text{Chains}(s) \setminus \text{Corrupted}(s)$, and position $k \in [0, \kappa - 1]$, define:

- $I_k^j(s) := \{x \in \mathcal{H} : \mathcal{A} \text{ queried } \text{Th}(P, T_{k+1}^j, x) \text{ in the first } s \text{ queries}\}$
- $O_k^j(s) := \{y \in \mathcal{H} : \exists x, \mathcal{A} \text{ queried } \text{Th}(P, T_k^j, x) \text{ with output } y \text{ in the first } s \text{ queries}\}$
- $IO_k^j(s) := I_k^j(s) \cup O_k^j(s)$

Note: $O_0^j(s) = \emptyset$, and Chain Oracle queries do *not* contribute to these sets.

Define the “found” events inductively:

- $\text{Found}_j^k(s) := (Y_k^j \in IO_k^j(s)) \cup \text{Found}_j^{k-1}(s)$, with $\text{Found}_j^{-1}(s) := \emptyset$
- $\text{Found}_j(s) := \bigcup_{k=0}^{\kappa-1} \text{Found}_j^k(s)$
- $\text{Found}(s) := \bigcup_{j \in \text{Chains}(s)} \text{Found}_j(s)$

For corrupted chains $j \in \text{Corrupted}(s)$, we define $\text{Found}_j(s)$ as the event that chain j was “found” before corruption: $\exists s' < s$ with $j \in \text{Chains}(s') \setminus \text{Corrupted}(s')$ and $\text{Found}_j(s')$ holds.

Finally, define:

$$\text{Guessed} := (\mathcal{A} \text{ wins}) \cap \neg \text{Found}(s_f).$$

Since $\mathcal{A}$ wins implies either $\text{Found}(s_f)$ or Guessed , we have:

$$\text{Adv}_{\text{Th}, \kappa, p}^{\text{GAME-2}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins}] \leq \Pr[\text{Found}(s_f)] + \Pr[\text{Guessed}]. \quad (1)$$

Step 2: Uniformity claim.

Claim 4.5 (Uniformity). *Let $\text{View}(s)$ denote $\mathcal{A}$'s view after s queries. For each $s \in [0, s_f]$, non-corrupted chain $j \in \text{Chains}(s) \setminus \text{Corrupted}(s)$, and $k \in [0, \kappa - 1]$:*

$$Y_k^j \mid (\text{View}(s), \neg \text{Found}(s)) \sim \text{Uniform}(\mathcal{H} \setminus IO_k^j(s)).$$

Proof of Claim 4.5. We proceed by induction on s .

Base case ($s = 0$): No queries have been made. $\text{Chains}(0) = \emptyset$, so the statement is trivially true.

Inductive step: Assume the claim holds after s queries. Consider the $(s + 1)$ th query. We analyze three cases depending on the query type.

Case 1: RO query $\text{Th}(P, T, x)$ with output y .

Fix a non-corrupted chain $j \in \text{Chains}(s) \setminus \text{Corrupted}(s)$ and position $k \in [0, \kappa - 1]$. Note that $\text{Chains}(s + 1) = \text{Chains}(s)$ and $\text{Corrupted}(s + 1) = \text{Corrupted}(s)$.

- If $T \notin \{T_k^j, T_{k+1}^j\}$: The sets I_k^j, O_k^j, IO_k^j are unchanged; distribution of Y_k^j unchanged.

- If $T = T_{k+1}^j$ (**query input** could match Y_k^j):
 - If $x = Y_k^j$: Triggers $\text{Found}_j^k(s+1)$. The claim becomes trivially true.
 - If $x \neq Y_k^j$: By induction, Y_k^j was uniform over $\mathcal{H} \setminus IO_k^j(s)$. Conditioning on $x \neq Y_k^j$, we get Y_k^j uniform over $\mathcal{H} \setminus IO_k^j(s+1)$ since $I_k^j(s+1) = I_k^j(s) \cup \{x\}$.
- If $T = T_k^j$ and $k \geq 1$ (**query output** could match Y_k^j):
 - If $y = Y_k^j$: Triggers $\text{Found}_j^k(s+1)$. The claim becomes trivially true.
 - If $y \neq Y_k^j$: By induction, Y_k^j was uniform over $\mathcal{H} \setminus IO_k^j(s)$. Conditioning on $y \neq Y_k^j$, we get Y_k^j uniform over $\mathcal{H} \setminus IO_k^j(s+1)$ since $O_k^j(s+1) = O_k^j(s) \cup \{y\}$.

Case 2: Chain Oracle query creating a new chain j' .

A new chain j' is created with tweaks $T_1^{j'}, \dots, T_\kappa^{j'}$ (provided by $\mathcal{A}$): $\text{Chains}(s+1) = \text{Chains}(s) \cup \{j'\}$, and $\text{Corrupted}(s+1) = \text{Corrupted}(s)$.

For existing chains $j \neq j'$: The sets $I_k^j(s+1) = I_k^j(s)$, $O_k^j(s+1) = O_k^j(s)$, and $IO_k^j(s+1) = IO_k^j(s)$ are unchanged (no RO query was made by $\mathcal{A}$). The internal RO evaluations use tweaks $T_1^{j'}, \dots, T_\kappa^{j'}$ that are distinct from all tweaks $T_1^j, \dots, T_\kappa^j$. Thus learning the output $Y_\kappa^{j'}$ provides no information about the values $Y_0^j, \dots, Y_{\kappa-1}^j$ of chain j . The distribution of Y_k^j conditioned on $\neg \text{Found}_j^k(s+1)$ remains uniform over $\mathcal{H} \setminus IO_k^j(s+1)$.

For the new chain j' : $\mathcal{A}$ may have previously queried Th with these tweaks, i.e. the set $IO_k^{j'}(s+1)$ may be non-empty for some $k \in [0, \kappa-1]$.

For $k = 0$: $Y_0^{j'}$ is sampled by challenger uniformly at random, and conditioning on $Y_0^{j'} \notin IO_0^{j'}(s+1)$ (due to $\neg \text{Found}_{j'}^0(s+1)$) gives uniformity over $\mathcal{H} \setminus IO_0^{j'}(s+1)$. For a consecutive value of $k \geq 1$: assuming $\neg \text{Found}_{j'}^{k-1}(s+1)$, we have $Y_{k-1}^{j'} \notin IO_{k-1}^{j'}(s+1)$, so $Y_k^{j'} = \text{Th}(P, T_k^{j'}, Y_{k-1}^{j'})$ is a fresh RO output. Conditioning on $Y_k^{j'} \notin IO_k^{j'}(s+1)$ gives the desired uniformity.

Finally, the adversary learns $Y_\kappa^{j'}$. We claim this does not bias the distribution of $Y_0^{j'}, \dots, Y_{\kappa-1}^{j'}$ beyond the conditioning already applied. To see this, note that for a random function f and X uniform over a set S (with X independent of f), the conditional distribution $X \mid f(X)$ remains uniform over S : by symmetry of f , all $x \in S$ are equally likely to map to any given output. Applying this to $Y_{\kappa-1}^{j'}$ (uniform over $\mathcal{H} \setminus IO_{\kappa-1}^{j'}(s+1)$ by the above) and the fresh evaluation $Y_\kappa^{j'} = \text{Th}(P, T_\kappa^{j'}, Y_{\kappa-1}^{j'})$, we see that revealing $Y_\kappa^{j'}$ preserves the uniform distribution of $Y_{\kappa-1}^{j'}$. The same reasoning applies inductively to earlier chain values.

Case 3: Corruption Oracle query on chain j' .

Chain j' becomes corrupted: $\text{Corrupted}(s+1) = \text{Corrupted}(s) \cup \{j'\}$, and $\text{Chains}(s+1) = \text{Chains}(s)$. The adversary learns all values $(Y_0^{j'}, \dots, Y_\kappa^{j'})$.

For chains $j \neq j'$: The sets I_k^j, O_k^j, IO_k^j are unchanged (no RO query was made). Since Th is a random oracle and the tweaks $T_1^{j'}, \dots, T_\kappa^{j'}$ are distinct from $T_1^j, \dots, T_\kappa^j$, learning the values $Y_0^{j'}, \dots, Y_\kappa^{j'}$ provides no information about $Y_0^j, \dots, Y_{\kappa-1}^j$. The distribution of Y_k^j conditioned on $\neg \text{Found}_j^k(s+1)$ remains uniform over $\mathcal{H} \setminus IO_k^j(s+1)$.

For chain j' : Chain j' is now in $\text{Corrupted}(s+1)$, so the claim statement no longer applies to it.

This completes the induction. □

Step 3: Bounding $\Pr[\text{Guessed}]$.

Claim 4.6. $\Pr[\text{Guessed}] \leq \frac{2}{|\mathcal{H}| - q_H}.$

Proof of Claim 4.6. Suppose $\mathcal{A}$ outputs (j^*, x) with $\neg \text{Found}_{j^*}(s_f)$. In particular, $Y_{\kappa-1}^{j^*} \notin IO_{\kappa-1}^{j^*}(s_f)$.

If $\mathcal{A}$ previously queried $\text{Th}(P, T_{\kappa}^{j^*}, x) = y$: since $Y_{\kappa}^{j^*} \neq y$ (else $\text{Found}_{j^*}^{\kappa-1}$ would hold), and $\mathcal{A}$ loses.

If $\mathcal{A}$ never queried $\text{Th}(P, T_{\kappa}^{j^*}, x)$: $\mathcal{A}$ wins only if $\text{Th}(P, T_{\kappa}^{j^*}, x) = Y_{\kappa}^{j^*}$. This requires either:

- (a) $x = Y_{\kappa-1}^{j^*}$: By Claim 4.5, $\Pr[(a)] \leq 1/(|\mathcal{H}| - q_H)$.
- (b) $x \neq Y_{\kappa-1}^{j^*}$ but the fresh RO output equals $Y_{\kappa}^{j^*}$: $\Pr[(b)] \leq 1/|\mathcal{H}|$.

Thus $\Pr[\text{Guessed}] \leq \frac{1}{|\mathcal{H}| - q_H} + \frac{1}{|\mathcal{H}|} \leq \frac{2}{|\mathcal{H}| - q_H}$. $\square$

Step 4: Bounding $\Pr[\text{Found}(s_f)]$.

Claim 4.7. $\Pr[\text{Found}(s_f)] \leq \frac{2q_H}{|\mathcal{H}| - q_H}$.

Proof of Claim 4.7. Define $\text{Transition}(s) := \neg \text{Found}(s-1) \cap \text{Found}(s)$. Since $\text{Found}(0) = \emptyset$:

$$\Pr[\text{Found}(s_f)] \leq \sum_{s=1}^{s_f} \Pr[\text{Transition}(s)].$$

We analyze each query type:

RO query at step s : Consider the query $\text{Th}(P, T, x) \rightarrow y$. By tweak distinctness across chains and positions, T can match at most one pair (j, k) where $j \in \text{Chains}(s) \setminus \text{Corrupted}(s)$ and $k \in [1, \kappa]$ with $T = T_k^j$. If no such match exists, the query cannot trigger Found . Otherwise, the query triggers $\text{Found}_j^{\kappa-1}(s+1)$ if either:

- $x = Y_{k-1}^j$ (same preimage): By Claim 4.5, this happens with probability at most $1/(|\mathcal{H}| - q_H)$.
- $x \neq Y_{k-1}^j$ but $y = Y_{k-1}^j$ (second preimage): Either the query is a duplicate (and nothing happens), or it's a fresh RO query and the output y is uniform over $\mathcal{H}$, so collision happens with probability $1/|\mathcal{H}|$.

Thus $\Pr[\text{Transition}(s) \mid s\text{-th query is RO}] \leq 1/(|\mathcal{H}| - q_H) + 1/|\mathcal{H}| \leq 2/(|\mathcal{H}| - q_H)$.

Chain Oracle query at step s creating chain j' : For the “same preimage” events: $Y_0^{j'}$ is sampled uniformly at random from $\mathcal{H}$ by the challenger. Except with probability $|I_0^{j'}(s)|/|\mathcal{H}|$, we have $Y_0^{j'} \notin I_0^{j'}(s)$, meaning $Y_0^{j'}$ was not previously queried at tweak $T_1^{j'}$. In that case, $Y_1^{j'} = \text{Th}(P, T_1^{j'}, Y_0^{j'})$ is a fresh RO output, uniform over $\mathcal{H}$. Continuing inductively: except with probability $|I_k^{j'}(s)|/|\mathcal{H}|$, the value $Y_k^{j'}$ is novel (not in $I_k^{j'}(s)$), and then $Y_{k+1}^{j'}$ is a fresh uniform output.

For the “second preimage” events: the set $O_k^{j'}(s)$ consists of outputs from prior queries at tweak $T_k^{j'}$, which are independent of $Y_k^{j'}$. So $\Pr[Y_k^{j'} \in O_k^{j'}(s)] \leq |O_k^{j'}(s)|/|\mathcal{H}|$.

By union bound over all $k \in [0, \kappa-1]$ and both event types:

$$\Pr[\text{Transition}(s) \mid s\text{-th query is for Chain } j] \leq \sum_{k=0}^{\kappa-1} \frac{|I_k^{j'}(s)| + |O_k^{j'}(s)|}{|\mathcal{H}|} \leq \frac{2 \sum_{\ell=1}^{\kappa} Q_{T_{\ell}^{j'}}(s)}{|\mathcal{H}|},$$

where $Q_T(s)$ counts RO queries with tweak T in the first s steps.

Corruption Oracle query: Cannot trigger Found .

Summing contributions: Let $\mathcal{J} = \text{Chains}(s_f)$. By tweak distinctness across chains, the sets $\{T_1^j, \dots, T_{\kappa}^j\}_{j \in \mathcal{J}}$ are pairwise disjoint. We partition RO queries into three types:

- Type 1: tweak $T = T_\ell^j$ for some $j \in \mathcal{J}$, $\ell \in [1, \kappa]$, queried *after* chain j was created.
- Type 2: tweak $T = T_\ell^j$ for some $j \in \mathcal{J}$, $\ell \in [1, \kappa]$, queried *before* chain j was created.
- Type 3: tweak $T \notin \bigcup_{j \in \mathcal{J}} \{T_1^j, \dots, T_\kappa^j\}$ (does not match any chain).

Let N_1, N_2, N_3 denote the counts of each type. By disjointness, each RO query falls into exactly one type, so $N_1 + N_2 + N_3 = q_H$.

Type 1 contribution: By the RO query analysis, each such query contributes at most $2/(|\mathcal{H}| - q_H)$ to $\Pr[\text{Transition}(s)]$. Summing over all N_1 such queries:

$$\text{Type 1 contribution} \leq \frac{2N_1}{|\mathcal{H}| - q_H}.$$

Type 2 contribution: Type 2 queries contribute when their corresponding chain is created. For each chain $j \in \mathcal{J}$, let s_j denote the step at which j is created.

The quantity $\sum_{\ell=1}^{\kappa} Q_{T_\ell^j}(s_j - 1)$ counts exactly the Type 2 queries whose tweak belongs to chain j (i.e., RO queries with tweak in $\{T_1^j, \dots, T_\kappa^j\}$ made before step s_j). Since the tweak sets are disjoint across chains, summing over all $j \in \mathcal{J}$:

$$\text{Type 2 contribution} \leq \sum_{j \in \mathcal{J}} \frac{2 \sum_{\ell=1}^{\kappa} Q_{T_\ell^j}(s_j - 1)}{|\mathcal{H}|} = \frac{2N_2}{|\mathcal{H}|}.$$

Type 3 contribution: Type 3 queries have tweaks that never match any chain, so they cannot trigger **Found**.

Total:

$$\Pr[\text{Found}(s_f)] \leq \frac{2N_1}{|\mathcal{H}| - q_H} + \frac{2N_2}{|\mathcal{H}|} \leq \frac{2(N_1 + N_2)}{|\mathcal{H}| - q_H} \leq \frac{2q_H}{|\mathcal{H}| - q_H}.$$

□

Conclusion. Combining (1) with Claims 4.6 and 4.7:

$$\text{Adv}_{\text{Th}, \kappa, p}^{\text{GAME-2}}(\mathcal{A}) \leq \Pr[\text{Found}(s_f)] + \Pr[\text{Guessed}] \leq \frac{2q_H}{|\mathcal{H}| - q_H} + \frac{2}{|\mathcal{H}| - q_H} = \frac{2q_H + 2}{|\mathcal{H}| - q_H}.$$

□

References

- [1] J. Drake, D. Khovratovich, M. Kudinov, and B. Wagner, “Hash-based multi-signatures for post-quantum ethereum,” Cryptology ePrint Archive, Paper 2025/055, 2025. [Online]. Available: <https://eprint.iacr.org/2025/055>